Software Requirements Specification

AgriChain

A transparent supply-chain tracking & margin analytics platform for agricultural products

Version 1.0

Date: February 22, 2026

Prepared for:
Jawaharlal Nehru Engineering College

Department of Computer Science and Engineering

Prepared by:

1. **Ganesh Deokar** – 202301103010 - Third-Year CS (Batch A1, TYCSE1)
2. **Ashmit Bhusare** – 202301103011 - Third-Year CS (Batch A1, TYCSE1)
3. **Sarthak Gosavi** – 202301103030 - Third-Year CS(Batch A2,TYCSE1)
4. **Vinit Singare** – 202301103032 - Third-Year CS (Batch A2, TYCSE1)

Table of Contents

1. Introduction 3
2. Overall Description 4
3. System Features (Functional Requirements) 7
4. External Interface Requirements 9
5. Non-Functional Requirements 9
6. Data Dictionary / Database Schema 10
7. Requirements Traceability Matrix (RTM) 12
8. Conclusion 13
9. Citations and References 13

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to provide a comprehensive, detailed architectural and functional blueprint for the "AgriChain" platform. This document will guide the engineering team in developing a minimum viable product (MVP) and serve as a benchmark for QA testing. It covers the full lifecycle of agricultural produce tracking from farm origination to consumer QR code scanning, utilizing a centralized relational database for reliable data tracking and transparency.

1.2 Document Conventions

This document adheres to the IEEE 830-1998 standard. Requirements are categorized by priority:

- **Must Have (Priority 1):** Critical for core supply chain tracking and database logging. 3
- **Should Have (Priority 2):** Analytics, AI Fairness Score, and government dashboards.
- **Could Have (Priority 3):** SMS integration for offline farmers.

1.3 Intended Audience and Reading Suggestions

- **Developers/Engineers:** Focus heavily on Sections 4 (System Features), 5 (NonFunctional Requirements), and the Data Dictionary.
- **Project Evaluators/Jury:** Focus on Section 1, Section 2 (Overall Description), Section 7 (RTM), and the Conclusion.

1.4 Product Scope

AgriChain is an end-to-end web and mobile-responsive application. It digitizes the agricultural supply chain to expose intermediary profit margins and ensure fair compensation for farmers. The system relies on a Node.js backend to enforce strict CRUD operations in a PostgreSQL database, ensuring that price entries are chronologically tracked and easily accessible for margin analytics.

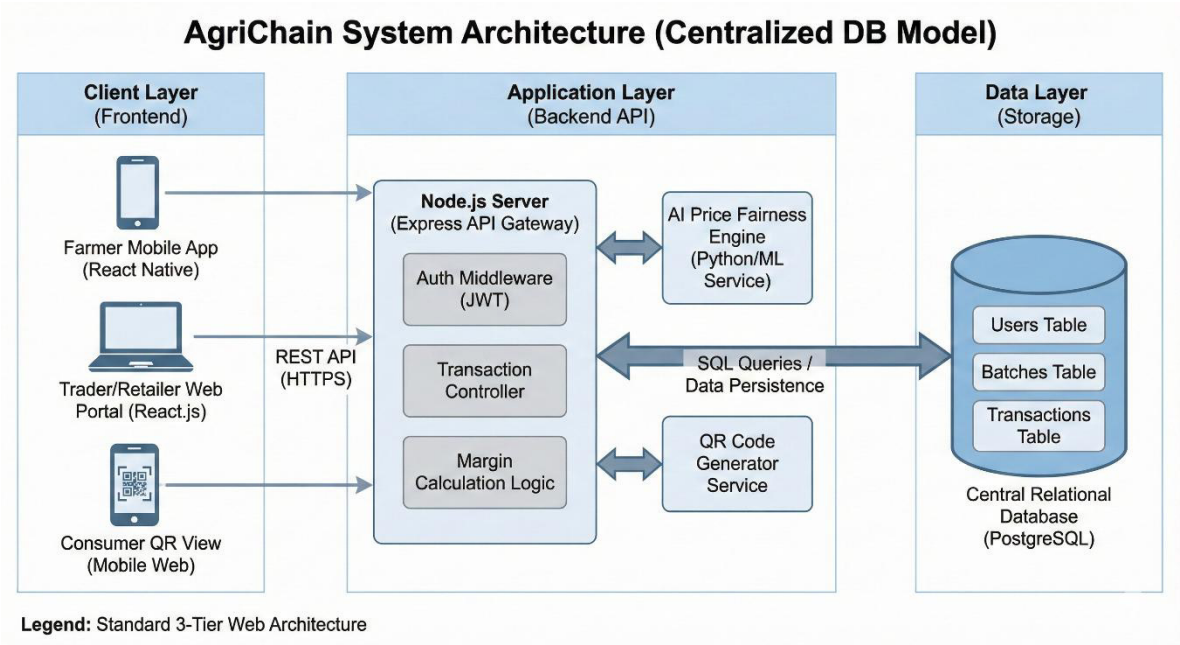
1.5 Definitions and Acronyms

- **RBAC:** Role-Based Access Control.
- **CRUD:** Create, Read, Update, Delete operations in a database.
- **RDBMS:** Relational Database Management System (e.g., PostgreSQL).
- **MSP:** Minimum Support Price (Government set baseline).
- **JWT:** JSON Web Token for stateless authentication.
- **RTM:** Requirements Traceability Matrix.

2. Overall Description

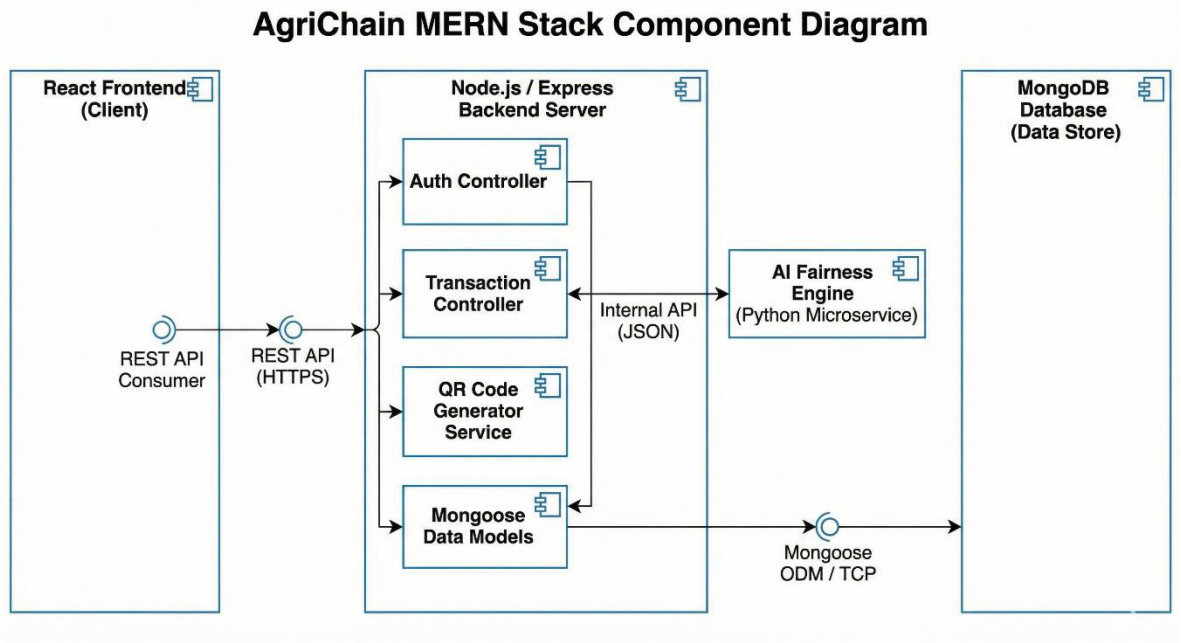
2.1 Product Perspective

AgriChain is designed as a standalone, monolithic application for the prototype phase, with an architecture designed to be easily decoupled into microservices for enterprise scaling. It operates entirely in the cloud, utilizing RESTful APIs for clientserver communication.



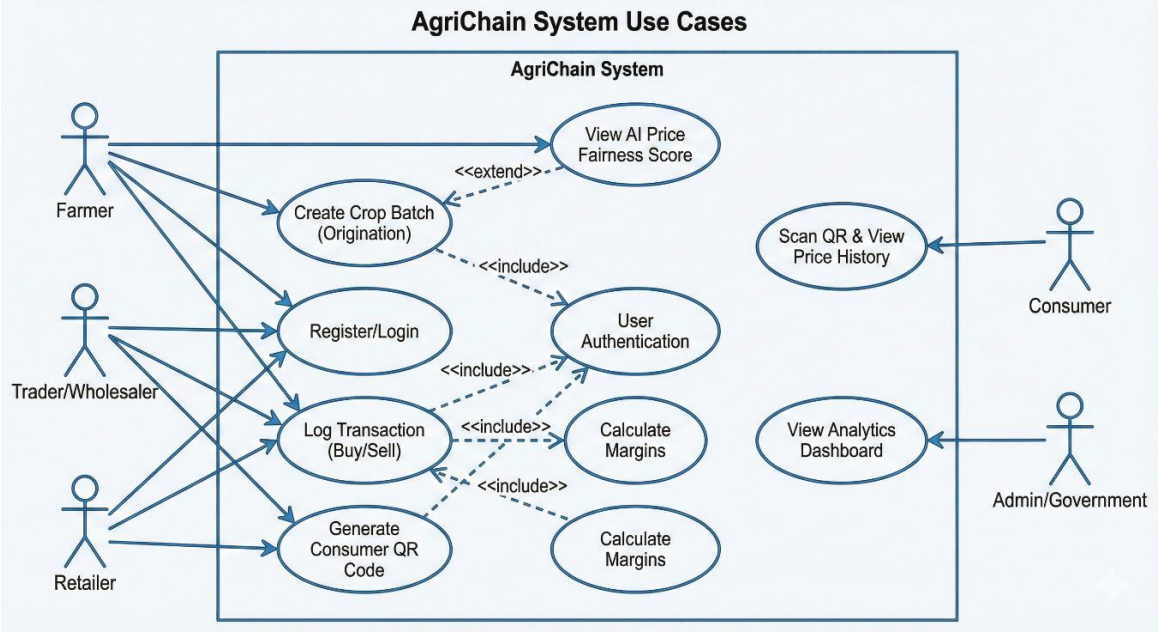
2.2 Product Functions

- **Identity Management:** Secure login and distinct dashboards for Farmers, Traders, Wholesalers, Retailers, and Consumers.
- **Digital Asset Origination:** The creation of a "Digital Twin" (Batch ID) for a physical harvest.
- **Supply Chain Transaction Log:** Sequential database logging of the transfer of ownership and price jumps.
- **Margin Analytics Engine:** Automated calculation of absolute and percentagebased margins at each node.
- **AI Price Fairness Calculator:** A Python-based microservice or integrated logic that cross-references current transaction prices with historical/MSP data.
- **Consumer Transparency Portal:** A frictionless, no-login required web view accessed via QR scan.



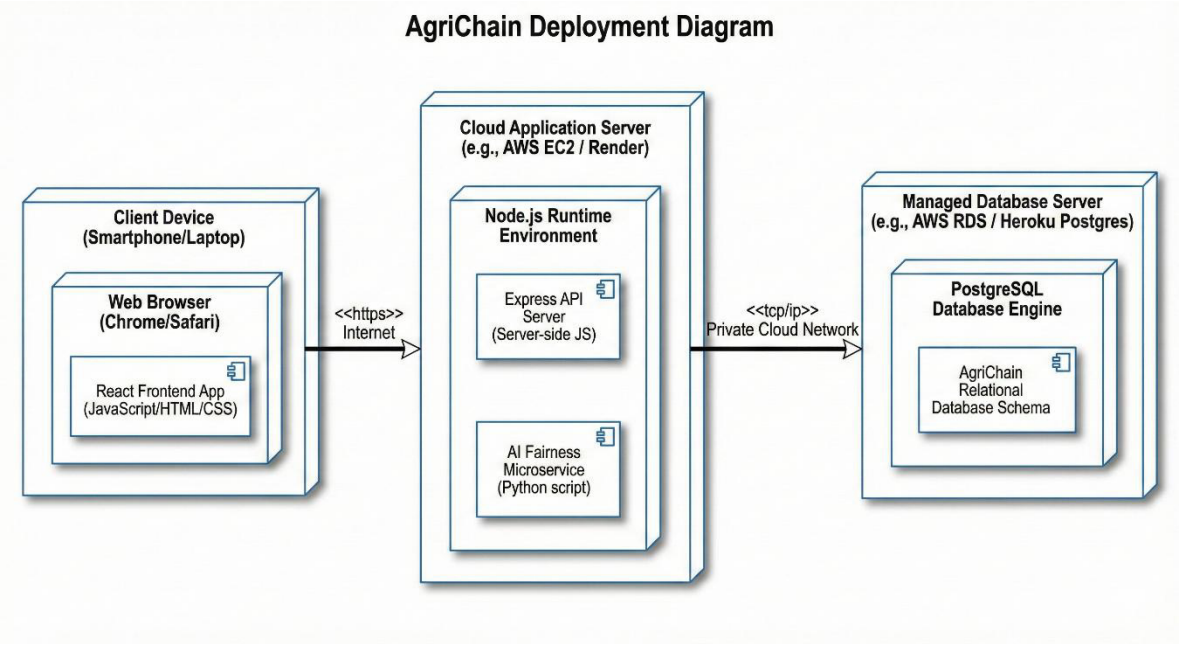
2.3 User Classes and Characteristics

- **Farmers (Originators):** Expected to have minimal technical literacy. Interface must be highly visual, native-language supported, and require minimal keystrokes.
- **Intermediaries (Traders/Wholesalers):** High-volume users. Require bulk entry capabilities, quick search/filter by Batch ID, and profit tracking dashboards.
- **Retailers (Endpoints):** Require POS (Point of Sale) style interfaces. Need quick generation of QR codes for physical printing or digital display.
- **Consumers (Viewers):** High technical literacy but low attention span. Require immediate, scannable data visualization upon scanning the QR code.
- **Government/Admins (Monitors):** Require macro-level data aggregation, anomaly detection alerts, and geographic margin mapping.



2.4 Operating Environment

- **Client Side:** Modern web browsers (Chrome v90+, Safari v14+, Firefox v88+). Mobile responsive down to 360px width. 6
- **Server Side:** Node.js v18.x LTS or higher runtime environment.
- **Database:** PostgreSQL v14 or higher.



2.5 Design and Implementation Constraints

- **Data Integrity Constraint:** Standard UPDATE or DELETE SQL commands on the Transactions table must be strictly disabled at the application level to prevent post-transaction price tampering by intermediaries.
- **Connectivity:** The application must account for potential data entry in lownetwork rural areas.

2.6 Assumptions and Dependencies

Assumptions:

- Users (Farmers/Retailers) possess smartphones with at least 3G/4G connectivity.
- MSP data is publicly available via API or static load. 7
- Retailers have hardware for QR code presentation.

Dependencies:

- Reliance on cloud infrastructure (AWS/Render).
- Dependency on open-source Node `qrcode` library.

3. System Features

3.1 Feature 1: User Authentication & RBAC

3.1.1 Description: Securely registers and authenticates users, assigning them specific roles.

3.1.2 Main Flow: User submits registration details -> System hashes password using bcrypt -> System issues a JWT upon successful login.

3.1.3 Error Handling: Return HTTP 409 for duplicate contact info; HTTP 401 for invalid credentials.

3.2 Feature 2: Harvest Origination (Batch Creation)

3.2.1 Description: Farmers log a new harvest, creating the initial database record for a specific batch.

3.2.2 Main Flow: Farmer inputs Crop Type, Quantity, and Selling Price -> System generates Batch_ID -> System records GPS coordinates -> Creates entry in Batches table.

3.2.3 Error Handling: Manual district selection if GPS fails; Reject negative quantities.

3.3 Feature 3: Transaction Tracking Engine

3.3.1 Description: The core engine tracking supply chain handoffs.

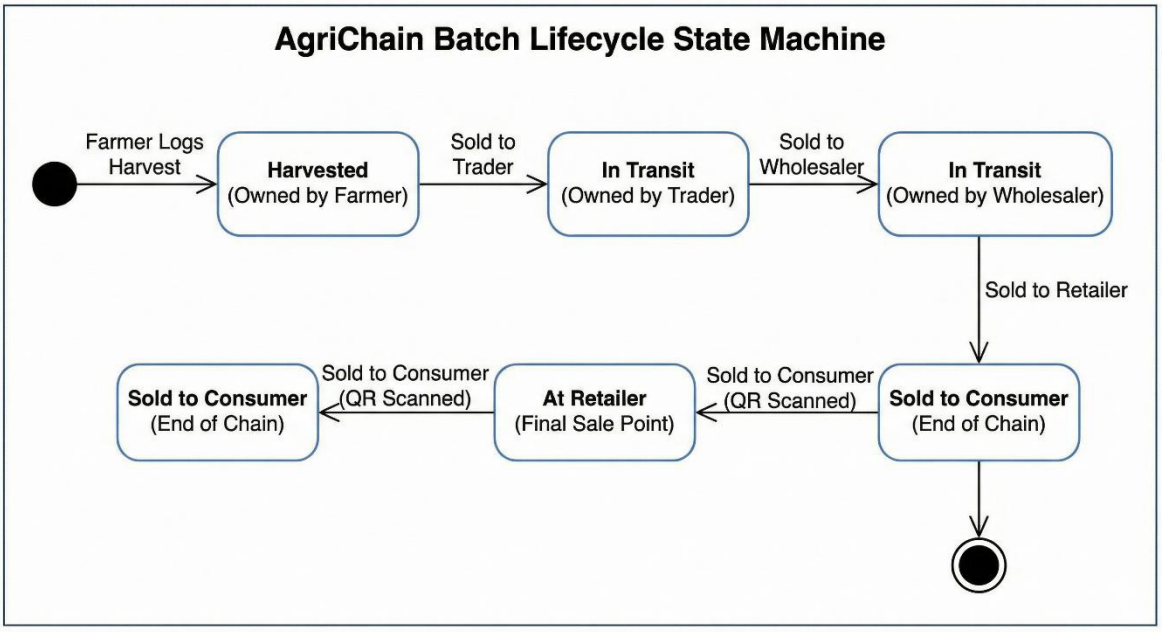
3.3.2 Main Flow: Trader enters Batch_ID and prices -> Backend validates request -> INSERTs a new row into the Transactions table.

3.3.3 Error Handling: HTTP 400 if user input buy_price != previous sell_price; Reject unauthorized ownership transfers.

3.4 Feature 4: AI Price Fairness Score

3.4.1 Description: A data-driven tool to empower farmers before they sell.

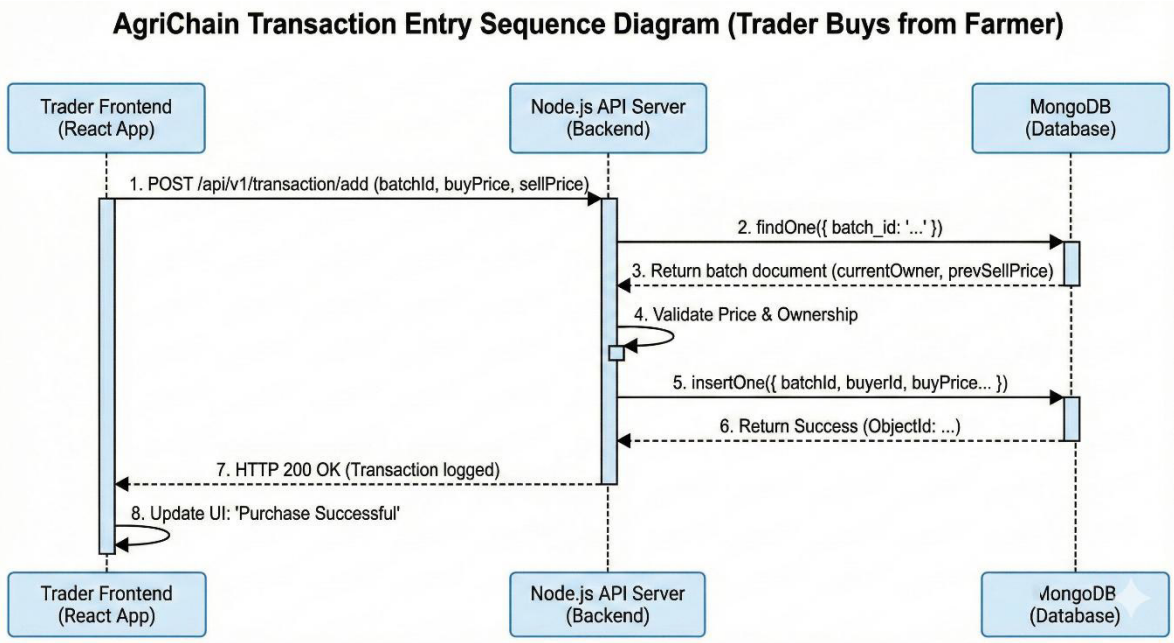
3.4.2 Main Flow: System queries retail price/MSP -> ML script processes variables -> UI displays "Fairness Score" (1-10).

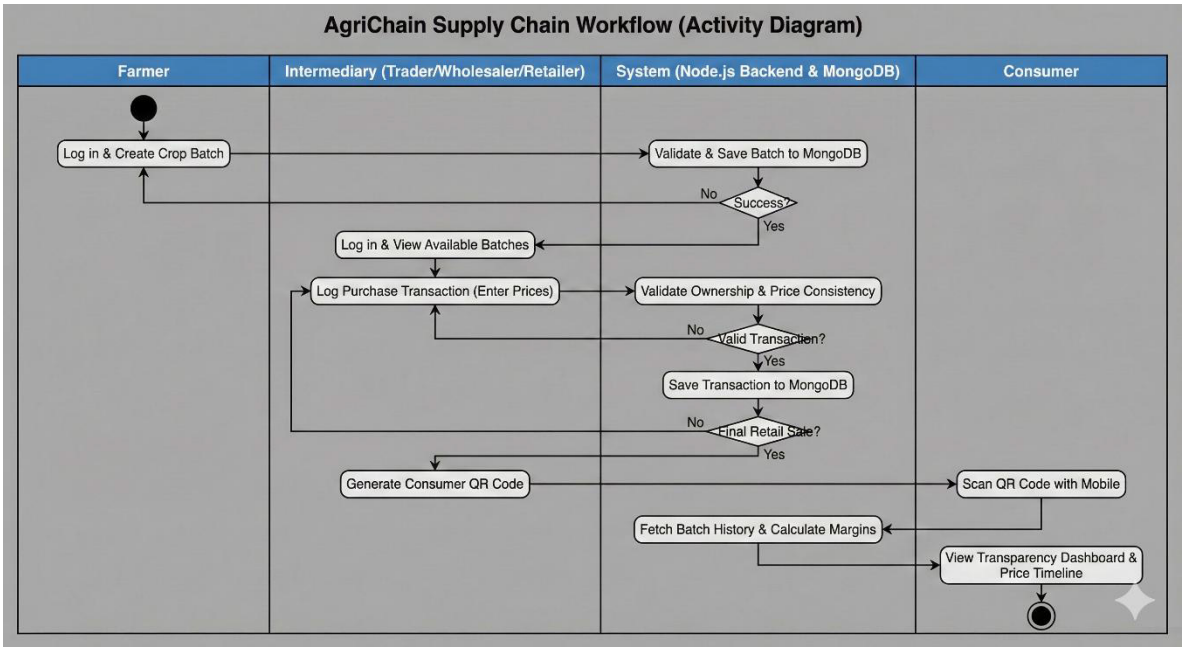


3.5 Feature 5: Consumer QR Code Transparency Portal

3.5.1 Description: The endpoint of the supply chain accessed via QR scan.

3.5.2 Main Flow: Retailer generates QR -> Consumer scans -> App fetches timeline -> Calculates Farmer Margin % -> Renders visual timeline.





4. External Interface Requirements

4.1 User Interfaces

- **Farmer Dashboard:** Minimalist UI with high-contrast, icon-driven navigation. 10
- **Consumer Trace UI:** Mobile-first vertical timeline view with price jump animations.

4.2 API Specifications (REST)

- `POST /api/v1/auth/register` - Registers a new stakeholder.
- `POST /api/v1/batch/create` - Creates initial batch record.
- `POST /api/v1/transaction/add` - Appends a transaction event.
- `GET /api/v1/trace/:batchId` - Returns full timeline for QR frontend.

5. Non-Functional Requirements

5.1 Performance

- **API Latency:** Trace endpoint must respond in under 300ms.
- **Query Optimization:** B-Tree indexes on `batch_id` in Transactions table.

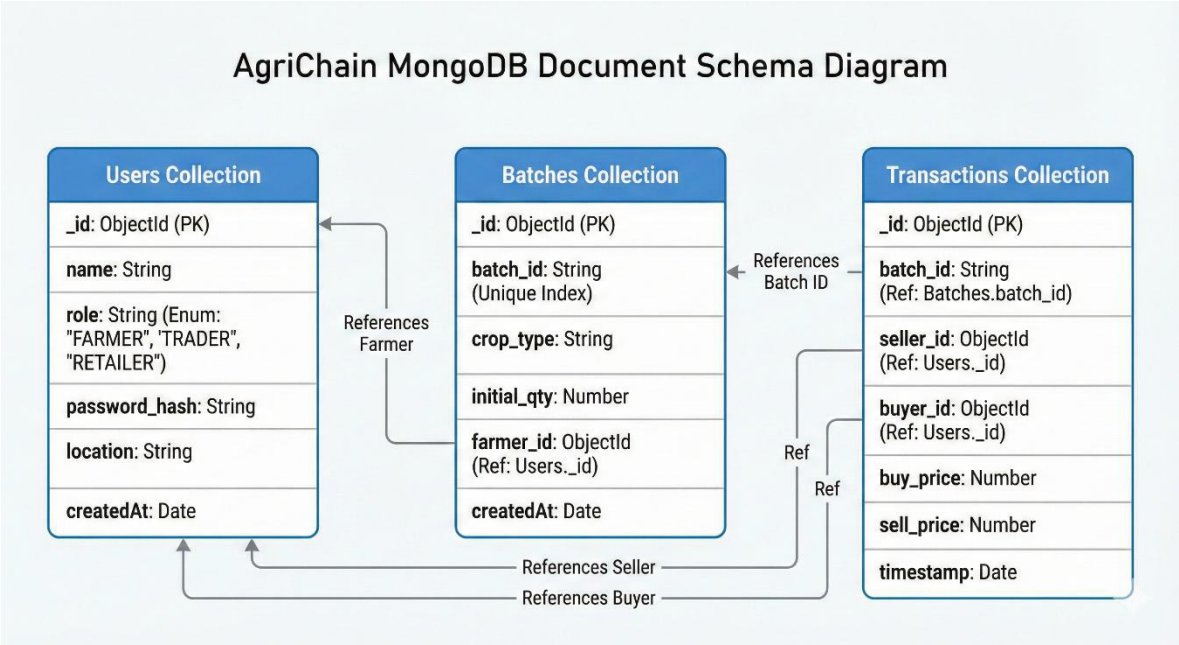
5.2 Security

- **Access Control:** All routes (except trace) protected by JWT.
- **Injection Protection:** Parameterized SQL queries for all PostgreSQL operations.

5.3 Reliability

- System designed for 99.9% uptime.
- Daily automated database snapshots.

6. Data Dictionary / Database Schema



6.1 Table: Users

Column Name	Data Type	Constraints	Description
user_id	UUID	Primary Key	Unique identifier.
name	VARCHAR(100)	Not Null	Full name or Business name.
role	ENUM	Not Null	'FARMER', 'TRADER', 'RETAILER'
password_hash	VARCHAR(255)	Not Null	Bcrypt hashed password.

6.2 Table: Batches

Column Name	Data Type	Constraints	Description
batch_id	VARCHAR(50)	Primary Key	Unique ID (e.g., ON-2026-001).
crop_type	VARCHAR(50)	Not Null	e.g., Onion, Tomato.
initial_qty	DECIMAL	Not Null	Quantity in kg.
farmer_id	UUID	Foreign Key	Links to Users table.
created_at	TIMESTAMP	Default NOW	Time of origination.

6.3 Table: Transactions

Column Name	Data Type	Constraints	Description
transaction_id	UUID	Primary Key	Unique ID for the step.
batch_id	VARCHAR(50)	Foreign Key	Links to Batches.
seller_id	UUID	Foreign Key	ID of the seller.
buyer_id	UUID	Foreign Key	ID of the buyer.
buy_price	DECIMAL	Not Null	Verified price paid by buyer.
sell_price	DECIMAL	Nullable	Price buyer intends to sell for.
timestamp	TIMESTAMP	Default NOW	Time of transaction entry.

7. Requirements Traceability Matrix (RTM)

Req ID	Description	User Role	Priority	Test Case ID / Method
REQ 1.1	User Registration & Password Hashing	All Roles	High	TC-AUTH-01: Verify bcrypt hash output in DB
REQ 1.2	JWT Token Generation on Login	All Roles	High	TC-AUTH-02: Validate token expiry logic
REQ 2.1	Generate Unique Batch ID	Farmer	High	TC-BATCH-01: Simulate concurrent inserts
REQ 3.1	Transaction Price Verification	Intermediary	High	TC-TRANS-01: Input mismatched prices
REQ 3.2	Enforce Sequential Ownership	System	High	TC-TRANS-02: Attempt sale by non-owner
REQ 4.1	AI Fairness Score Calculation	Farmer	Medium	TC-AI-01: Assert output score 1-10
REQ 5.1	QR Code Generation & Routing	Retailer	High	TC-QR-01: Scan QR, verify routing

8. Conclusion

The AgriChain platform addresses a critical socio-economic vulnerability within the agricultural sector: the lack of price transparency between the farm and the consumer table. By shifting the paradigm from an opaque, fragmented supply chain to a digitized, traceable, and immutable relational database architecture, AgriChain mitigates price manipulation and arbitrary inflation by intermediaries. Through the implementation of this Software Requirements Specification, the development team has a clear, robust blueprint to deliver a highly scalable MVP.

9. Citations and References

1. IEEE Computer Society. (1998). *IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998)*. IEEE Standards Association.
2. PostgreSQL Global Development Group. (2024). *PostgreSQL Documentation: Chapter 13. Concurrency Control*. 15
3. Node.js Foundation. (2024). *Node.js Documentation: Crypto Module*.
4. Tilkov, S., & Vinoski, S. (2010). *Node.js: Using JavaScript to Build High-Performance Network Programs*. IEEE Internet Computing.
5. Meta Platforms, Inc. (2024). *React: A JavaScript library for building user interfaces*.
6. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research.